R1 Re-baseline — Real Robot Runbook

Ten runs on the golden checkout, Gate 1 decided at the lab · every command paste-ready

What changed in rev. 2, and why. Rev. 1 was checked against the repository, the golden-window datasets and the lab machine itself. Six things moved; five of them change what you type.

1. **The runs happen ON THE LAB MACHINE, not on the laptop.** Every golden dataset carries `G1_PERC=127.0.0.1:8008`, the runs are stored there, and the USB proxy is installed there. Rev. 1 had it backwards (§0, §5).
2. **Three abort flags must NOT be passed.** The golden window ran on the binary's defaults (75 s / 0.4 m / 8 s). Rev. 1's line passed 150 / 0.25 / 16, making the supervisor twice as patient — which inflates the arrival rate Gate 1 measures (§5).
3. **The perception line starts with `G1_FLOORCOLOR=1`**, missing from every previous version of this document (§2.4).
4. **The shared-log sync is already done** (13 Aug). The lab machine had nothing to push (§2.1).
5. **Check GPU memory before launching perception**: a local LLM service holds ~20 GB per card on that box (§2.4).
6. **Golden is not byte-exact** and cannot be — the five clean runs span four commits, all with a dirty tree (§0).

Scope. Real robot only; the twin lane is deliberately out of this document. **The three iron rules.** One change per batch · No code edits during the session · Instrumentation down = run invalid, repeat it.

0. Choices this runbook already makes

QUESTION	CHOICE	BECAUSE
Which machine drives the robot	**The lab machine** (repo at `~/Documents/G1_UNITREE_ROBOT_META_REASONING`), iPhone by USB into it	**Resolved by evidence.** All nine golden-window datasets record `G1_PERC=127.0.0.1:8008`; the 24 Jul runs are stored on that machine (57 of them); the USB proxy is installed there and its shell history holds the exact run line. Driving from the laptop instead would introduce a new variable into the one batch whose whole purpose is comparability.
Water or dry	**Dry** — empty cup in the hand	The Disciplined Protocol states it outright: "*DRY (no water) … Next batch: WATER at 200 g on this exact binary*". Gate 1 measures arrivals, collisions and times — water contributes nothing to today's decision, and the QoE contract is not frozen yet.
Perception intrinsics	**Resolved**: `--cx 160 --cy 90`, with `G1_FLOORCOLOR=1`	The frames are 16:9 at 320×180, so `--cy 90` is the geometrically correct half-height; `--cy 120` is the older 4:3 assumption and appears earlier in the history. The

		`G1_FLOORCOLOR=1` prefix is in every line that box actually ran.
`--self-mask 0.25`	**Off** for this batch	A judgement, so write it down. It appears only in the most recent history line, and the apron work it belongs to came out of the post-collapse analysis — after the golden window. Reproducing golden argues for leaving it off. Server flags are not recorded in the datasets, so this cannot be proven either way.
`goto` or `gotoviz`	`gotoviz`	Same code path plus the visualisation window. The golden runs used `gotoviz` — it is in that machine's history verbatim.

> **Golden is a reference, not a byte-exact target.** The five clean crossings span four commits (`2522c11 → 53740e4 → 1e1904d → f312dd4 → 019ec85`) and **every one of them ran with a dirty working tree**. `019ec85` is the code of the last four runs, which makes it the best available reference — but exact reproduction is impossible by construction. That is precisely why this batch is ten runs and a threshold, not one run and an impression.

Pre-register the hypothesis (write it down before the first run)

> **H0:** with the golden binary, cup in hand, no apron — B→A success ≥ 70%, A→B ≥ 90%, times 50–110 s. If the batch does not reproduce this, the golden window was luck or something physical changed, and we will know it with *n*, not with vibes.

1. Who does what

ROLE	OWNS
Console	Launches each run, watches the run console, confirms the marker heartbeat line is there, calls REACHED/FAILED, pushes after each pair.
Marker & observer	Runs `spill_mark.py`: presses `i` the moment instrumentation drops, and keeps the per-run sheet — arrival or failure, time, collisions, near-misses. On a dry batch this seat is above all **the honest witness for what the robot does not register**: those events have no other record (§5).
Both	Nobody touches the robot during a run. A failed run is data, not a bug to fix live.

2. Pre-flight — before the robot is switched on

2.1 Shared-log sync — already done, 13 Aug 2026

> The lab machine was four commits behind with a 48 MB `goto.log` and **nothing of its own to push**. It has been pulled onto current `main`, its log is the rotated 4 KB one, and its old copy was archived compressed on that machine (4.8 MB) on top of the git history. **Nothing to do here today.** If a future session rotates again: the machine that rotated pushes first, the other pulls before its next run; conflicts in `goto.log` or the stats CSV are resolved by **union of both sides**, never by discarding lines.

2.2 Golden checkout — on the lab machine

```
cd ~/Documents/G1_UNITREE_ROBOT_META_REASONING && git status
```

Working tree must be clean (untracked scratch files are fine). Then:

```
git checkout golden-doorvis && git log --oneline -1
```

Expect `019ec85 "shakedown B->A con DOOR-VIS"`. Detached HEAD is fine for the session. The golden binary **includes** DOOR-VIS, DOOR-GUARD, DOOR-CENTER, REALCAL, marker v2 with heartbeat and the per-run config snapshot; it **does not include** RETREAT, SELF-MASK, ENV-CHANGE or OMNI. Those come back one at a time, twin-validated, after Gate 1.

2.3 Meta-reasoner sanity check

```
cd meta-reasoner-2.0 && python3 -m unittest discover -q
```

Must print OK. The Meta-Reasoner 2.0 is never modified; its 11 unit tests staying green is the guarantee.

2.4 Perception server — same machine

First, check the GPUs are free. A local LLM service lives on that box and can hold ~20 GB on *each* card, leaving too little for the depth model:

```
nvidia-smi --query-gpu=memory.used,memory.total --format=csv
```

If both cards are near full, stop that service before launching perception. Discovering this mid-session, with the robot on the floor, is the expensive way to find out.

Then launch it — this is the line that machine actually ran, with the two decisions from §0 applied (`--cy 90`, no self-mask):

```
source ~/g1env/bin/activate
G1_FLOORCOLOR=1 python perception_server.py --host 0.0.0.0 --port 8008 --fx 300 --fy 300 --cx 160
--cy 90 --cam-h 1.10 --cam-pitch -10 --debug
```

Verify:

```
curl -s http://127.0.0.1:8008/health
```

> **PASS = `ok: true` and MODE gpu.** If detections come back empty on the smoke test, the camera frame or the intrinsics are wrong. Stop and fix it before any run — a session recorded through a blind perception server is a wasted session.

2.5 Physical setup — frozen, and it goes into the manifest

- **Cup in the hand**, campaign-1 geometry (`ARM_R=0.25`). This keeps the cup out of the camera frame and the spill model calibrated. The chest mount broke both at once.
- **No apron** for the baseline batch. Water protection, if it comes back, is its own experiment.
- **Empty cup.** Scale and water stay at hand anyway: if the session goes well and the QoE contract gets frozen, the water batch is the immediate next step on this same binary.
- Phone with the vendor app, Bluetooth ON.

3. Robot bring-up (every boot)

Three terminals stay open all session: the USB proxy · the perception server · `spill_mark.py`.

1. On the phone, **"Forget This Network"** first. Then robot on → open the vendor app **with Bluetooth ON** → pair **from the app**: it reads the rotating key over BLE and joins the phone itself. Joining from iPhone Settings will say "incorrect password" — that is expected, do not fight it.
2. iPhone to the **lab machine** by USB → `ios_webkit_debug_proxy` sees the WebView → confirm the robot actually moves, in-app teleop or `g1_teleop.py`.
3. Localisation check:

   ```
   python3 g1_goto.py reloccheck
   ```

   The session ritual also runs `noisecheck` before this one.
4. Marker up in **its own terminal, before the first run**:

   ```
   python3 spill_mark.py
   ```

   Then verify its **heartbeat line appears in the run console**. The heartbeat is still unvalidated on hardware, so check it every session, not once. Leave the marker running *between* runs. If it dies mid-run that run is invalid: Ctrl-C, relaunch, redo the run.

> If the robot **reboots mid-session** the phone link breaks and the whole ritual starts again — and it must be **logged between runs**, because it is a session event. Same for the **app version**: it is an experiment variable. Note it, keep auto-updates off, keep a spare device on the old version.

> **Marker keys on the golden code (v2):** `ENTER` = spill · `w <g>` = weigh · `i` = invalid. The human-support keys `c` and `m <cm>` **do not exist here** — they are v3, on the feedback branch, and that branch is not part of R1. If someone expects them, they will be pressing keys that do nothing.

4. Shakedown — one or two runs, not scored

Before the batch starts, run the whole chain once end to end and **throw the result away**. It is the first real session in weeks and there are several things that can be quietly broken; finding them inside a scored run costs a run.

CHECK ON THE SHAKEDOWN RUN	WHAT A FAILURE LOOKS LIKE
Marker heartbeat in the run console	No heartbeat line → every subsequent run would be uninstrumented and invalid
Perception returning detections	Empty detections → wrong camera frame or intrinsics
Localisation holding	It was lost at the end of session 2 — `reloccheck` passing once is not the same as holding through a traverse
Door crossing at all	`door_crossed` never appears → DOOR-VIS is not doing what it did on 24 Jul
Nothing physical moved	Furniture, lighting, the B start pose. These are the H0-failure suspects, and they are cheaper to notice now

Log the shakedown as a shakedown. It is not run 1, it does not enter Gate 1, and it does not count towards the stop rule.

5. The run loop — 10 runs, 5× A→B and 5× B→A, alternating

Same command every single run. Any change mid-batch voids the batch.

1. Empty cup in the hand. The marker stays running for its heartbeat and for `i`.
2. Launch the run. This is the golden-window line, reconstructed from that machine's own history and from the environment snapshot stored inside the golden datasets, plus the physical manifest:

```
cd ~/Documents/G1_UNITREE_ROBOT_META_REASONING && source ~/g1env/bin/activate
G1_SETUP="cup=hand,apron=off,fill=dry" G1_ARM=REBASELINE G1_META2=2 G1_M2_CFG=config_meta2_g1paylo
ad.json G1_M2_STATE=meta2_state_r1.json G1_M2_STATE_INIT=meta2_state_twin_explored.json G1_M2_REAL
CAL=1 G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 G1_DOOR_VIS=1 G1_PE
RC=127.0.0.1:8008 python3 g1_goto.py gotoviz B
```

> **Do not add `G1_M2_ABORT_WIN`, `G1_M2_ABORT_PROG` or `G1_M2_HELP_S`.** Earlier drafts of this runbook passed 150 / 0.25 / 16. The golden window ran on the binary's **defaults — 75 s, 0.4 m, 8 s** — because those variables were simply not in its environment. Passing the larger values makes the supervisor roughly twice as patient, which turns runs that golden would have aborted into runs that continue: it inflates the very arrival rate Gate 1 is measuring. Leave them out.

`G1_SETUP` is not read by any code; the recorder snapshots every `G1_*` variable, so it lands in the dataset as the physical manifest of the batch. `G1_ARM=REBASELINE` is what separates these runs from the July ones in the analysis.

3. **During the run:** nobody touches the robot. Watch and note collisions and near-misses, especially against low furniture — see the table below.
4. **Run ends:** record arrival or failure, time, and collisions. A run that ends in a HELP stop is a **valid result**, not a failed run.
5. Anything wrong with the instruments → `i <reason>` and **repeat that run**. It is not a data point.
6. **Reverse leg:** exactly the same line, ending `g1_goto.py gotoviz A`.
7. Push after each pair and report the pair's outcomes: arrivals, times, collisions. Live accounting without touching the robot.

> **Stop rule.** Three consecutive failures of the same mode → **stop the session**. Data goes home, autopsy offline. No lab improvisation, no code edits. The 24 Jul collapse — five code edits and three physical changes in one afternoon — is exactly what this rule exists to prevent, and it cost an unattributable session.

What to expect, so nobody panics mid-session

SIGNAL	NORMAL ON THE REAL ROBOT
B→A leg time	52–60 s in the golden window; band ~50–110 s
Collisions	Median 0 per run
Spills	None to count on a dry batch. For the water batch that follows: **~12 per run at 247 g is NORMAL** — REALCAL handles it, and the twin's 0.5 is the simulator's physics, not a target
The B pocket	B starts ~0.3 m from a solid, "nose to sofa". Expect trouble on that side; it is a known waypoint problem, not a stack regression
Laser reading clear	Not evidence of clear space: 107 of 193 real collisions in replay happened with the laser reading clear beyond 0.6 m

What the robot physically cannot see

All ten collisions of an earlier session were attributed from the logs. Three mechanisms, and none of them is "the software had a bad day" — knowing them tells the observer where to stand and what to write down.

MECHANISM	WHAT ACTUALLY HAPPENS
Body model too thin 6 of 10	The planner assumes a half-width of 0.22 m (shoulders), but with arm swing the real robot is **0.28–0.30 m**. Door-jamb hits had the jamb correctly in the map and the planner advanced anyway.
Height-band blindness 2–3 of 10	The obstacle map only accepts laser returns in a band of roughly **0.7–1.8 m above the floor**. A couch arm or a low cabinet is simply invisible — the map read empty for 0.87 m around the impact point. The camera saw it the whole time and contributes almost no cells.
Thin jamb at a grazing angle 1–2 of 10	At grazing incidence the jamb returns too few points and the noise-persistence filter drops it intermittently — the hard-obstacle distance flapped between 0.7 and 2.5 m over the final three seconds before a hit.

For the observer: low furniture is the blind spot. If the robot is heading for a couch arm or a low cabinet, it does not know. Note it as an observed near-miss even when nothing is logged — that observation is the only record that will exist.

6. Gate 1 — decided at the lab, before packing up

CHECK	PASS
A→B reached	≥ 90% (≥ 5/5, or 9/10 counting both directions)
B→A reached	≥ 70% (≥ 4/5)
Collisions	Median 0 per run
Times	Golden band ~50–110 s (24 Jul reference: 52–65 s on B→A legs)

PASS → the baseline is re-established; the next sessions follow the Results Acquisition Plan (R2 water GOV/BASE ABBA → R3 → R4), and the meta-state-machine branch earns its own real trial, mirrored 1:1 against this batch.

FAIL → data home, autopsy offline with the analysis tools. Change nothing until the cause has a name. The suspects, in order: something physical moved, perception intrinsics, the B pocket.